Relatório de Auditoria de Segurança

AVASEC — Portal Escult / Escola Paulo Freire

Data	28 de agosto de 2026
Versão auditada	e2b8abf (branch chore/pendencias-pos-adr10)
Achados	1 crítica · 3 alta · 8 média · 3 baixa
Pontos fortes	21 controles verificados e aprovados

Escopo auditado

Frontend	React 19 + TypeScript, Vite 6, Tailwind CSS 4. SPA sem router: a navegação é feita por estado.
Backend	Laravel 13 (PHP 8.4), ORM Eloquent, MySQL 8. Controllers finos e regra de negócio nos services.
Autenticação	JWT HS256 próprio (firebase/php-jwt) em cookie HttpOnly 'ava_session', SameSite=Lax, validade de 12h. Senhas com password_hash/password_verify nativos (bcrypt).
Autorização	Middlewares 'jwt', 'active' e 'role:<perfis>', somados aos helpers App\Support\Identity, InstructorScope e CourseAccess, chamados dentro dos services.
Infraestrutura	docker-compose.yml apenas com o MySQL de desenvolvimento. Não há CI (nenhum .github/workflows), nem Helm, nem Terraform. A produção está documentada em DEPLOY_LARAVEL.md (Nginx + PHP-FPM).

Nota metodológica

Cada categoria do roteiro foi traduzida para o equivalente desta stack antes da leitura do código. Todo achado abaixo foi confirmado no código real e, quando indicado, no comportamento da aplicação em execução — nada foi inferido por semelhança.

1. Banco sem tranca (isolamento)
O projeto não usa RLS: o isolamento é feito em código, por filtro manual nas consultas, através de Identity::applyOwnRows (dono do registro), InstructorScope (instrutor restrito aos seus cursos e alunos) e CourseAccess (participante da turma). A auditoria procurou listagens, buscas e agregações que não aplicam nenhum desses filtros.

2. Permissão definida no navegador
Cada restrição de papel existente no frontend foi cruzada com a rota correspondente em routes/api.php e com o middleware RequireRole efetivamente resolvido (php artisan route:list -v), para verificar se o servidor repete a checagem que a interface faz.

3. IDOR
Os 72 handlers registrados foram percorridos um a um. Para cada rota que recebe um identificador (no caminho, na query ou no corpo), o service correspondente foi lido em busca de checagem de posse ou de escopo antes da leitura, da alteração ou da exclusão.

4. Chaves expostas
Varredura do código, das configurações, do docker-compose, de db/init, dos seeders, da documentação e dos 69 commits do histórico do git. Também foi inspecionado o pacote de produção (dist/) em busca de credenciais embarcadas e de valores padrão que se tornam segredo real quando não são sobrescritos.

5. Entradas sem tratamento (XSS)
No frontend, busca por dangerouslySetInnerHTML, innerHTML, eval/new Function e por URLs controladas pelo usuário em href e src. No backend, a validação dos campos de URL e a existência de biblioteca de sanitização — não há nenhuma no projeto.

Categorias sem aplicação nesta stack

RLS de banco, o padrão do Supabase: O projeto não usa Supabase nem políticas de linha no MySQL. O isolamento é feito em código, por Identity, InstructorScope e CourseAccess, e foi auditado nessa forma.

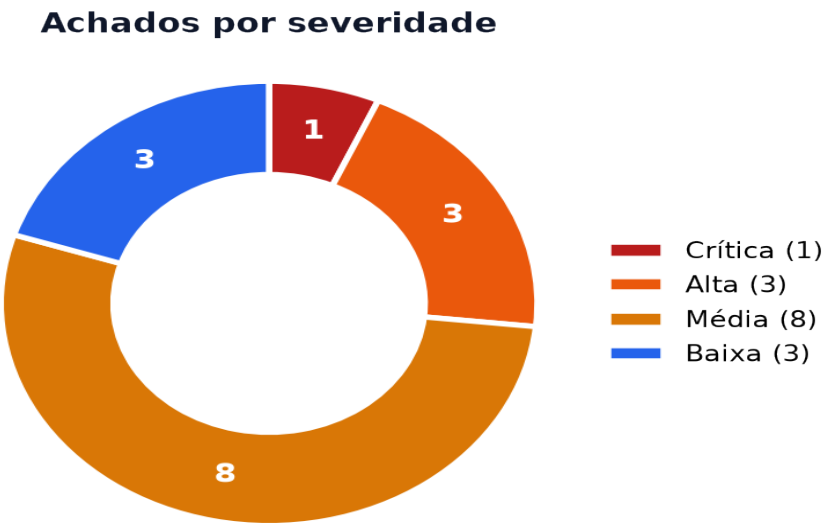
Segredos em CI/CD, Helm e Terraform: Não existem no repositório: não há .github/workflows, chart Helm nem arquivos .tf. O único artefato de infraestrutura é o docker-compose.yml do MySQL de desenvolvimento.

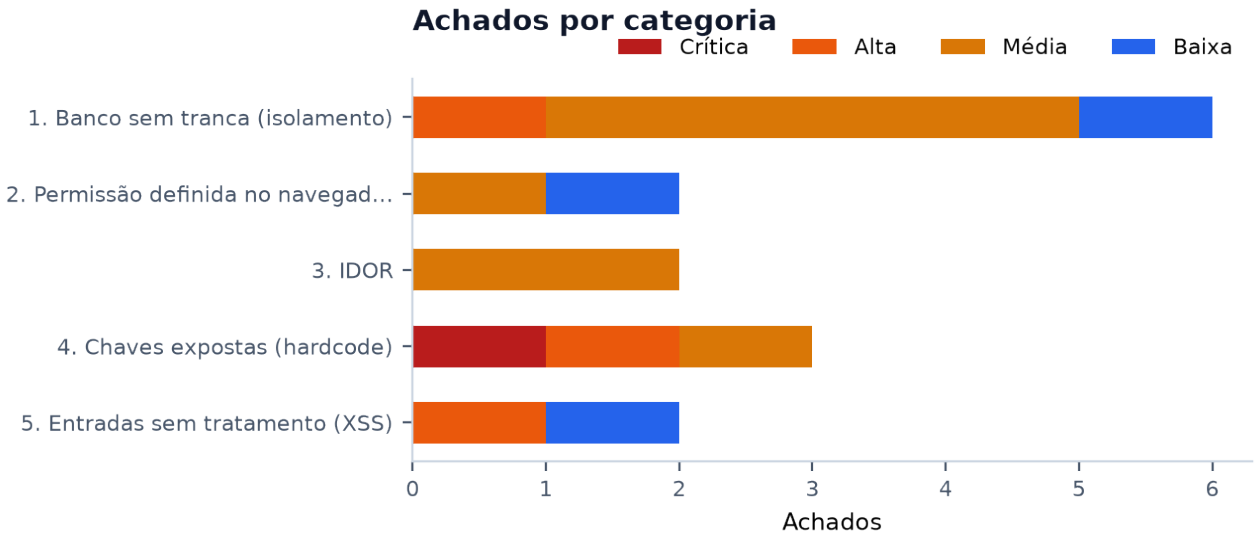
XSS em e-mails e em templates do servidor: A aplicação não envia e-mail — as variáveis MAIL_* não têm configuração real — e as respostas da API são JSON. O único template Blade renderizado é o do certificado, alimentado por campos já validados.

Resumo executivo

A auditoria percorreu os 72 handlers registrados no backend, o frontend e os artefatos de build, e registrou **15 achados**, dos quais **4** são de severidade alta ou crítica, além de **21 controles verificados e aprovados**. O achado dominante não está em uma regra de negócio, e sim na configuração de build: uma linha no .env faz o pacote de produção ser gerado em modo de desenvolvimento, publicando na tela de login as senhas de Admin e Gestor.

Severidade	Qtd.	Leitura
CRÍTICA	1	Exige correção imediata; explorável sem conta e sem pré-condição.
ALTA	3	Correção prioritária: dado sensível ou credencial exposta.
MÉDIA	8	Falha de escopo entre perfis legítimos; corrigir no ciclo corrente.
BAIXA	3	Risco contido ou dependente de ação da própria vítima.





Pontos fortes verificados

A lista abaixo não é genérica: cada item foi conferido no arquivo indicado e, em vários casos, existe teste automatizado cobrindo o comportamento. Serve também como prova da cobertura desta auditoria.

- ✔ **Identidade por chave estrangeira, não por nome (ADR 10)**
  App\Support\Identity centraliza a resolução de dono. Há teste dedicado provando que homônimos não enxergam os dados um do outro: EnrollmentTest::test_homonyms_do_not_leak_each_others_enrollment.

- ✔ **Nota de avaliação recalculada no servidor**
  LearningService::submitQuiz (127-165) ignora qualquer pontuação enviada pelo cliente e recalcula comparando com correctOptionIndex; o courseId vem do quiz, não do corpo. O aluno não autodeclara nota.

- ✔ **Frequência e certificado não forjáveis**
  EnrollmentService::sanitizeProgressIds (130-145) descarta identificadores de aula e de sessão que não pertencem ao curso e remove duplicatas — sem isso o aluno inflava a própria frequência. CertificateService::issueCertificate (106-113) recalcula a frequência antes de emitir.

- ✔ **Download de PDF de certificado com dupla checagem**
  CertificatePdfService::renderPdf (37-44): o aluno só baixa o próprio; o instrutor, apenas os de curso que leciona.

- ✔ **Chat de aula ao vivo restrito à turma**
  MessagingService::listChatMessages (31-52) e createChatMessage (59-75) validam CourseAccess::canAccess antes de ler ou escrever.

- ✔ **Mensagens diretas isoladas por dono e por escopo do instrutor**
  MessagingService::listDirectMessages (81-102): o aluno vê apenas o próprio canal; o instrutor fica limitado a InstructorScope::studentIds; o admin é irrestrito.

- ✔ **Exclusão de mensagem do fórum verifica posse**
  LearningService::deleteForumMessage (215-226) exige Identity::ownsRow ou o perfil admin.

- ✔ **Renomeação de usuário restrita ao próprio ou ao admin**
  AuthService::renameUser (263-267) rejeita com 403 antes de qualquer escrita e propaga o novo nome em transação para os campos de exibição.

- ✔ **Verificação pública de certificado com lista branca de campos**
  CertificateService::verifyCertificatePublic (52-83) devolve apenas campos públicos — a rota sem autenticação nunca expõe userId nem enrollmentId.

- ✔ **Limite de requisições por identidade, não por IP**
  AppServiceProvider::configureRateLimiters: o login conta por par (identificador, IP), com teto por IP; as rotas autenticadas contam por usuário do token. Isso evita que um laboratório de escola atrás de um único IP bloqueie o acesso da própria turma. O bloqueio de conta após cinco tentativas está em AuthService:121-135.

- ✔ **Proxy confiável declarado e IP de auditoria não forjável**
  bootstrap/app.php declara trustProxies apenas para 127.0.0.1 e ::1; AuditLogger::clientIp e AuditController usam $request->ip(). Há teste provando que um X-Forwarded-For de origem não confiável é ignorado: RateLimitKeyTest::test_audited_ip_cannot_be_forged_by_the_client.

- ✔ **Interrupção imediata do boot com JWT_SECRET fraco em produção**
  AppServiceProvider::assertStrongJwtSecret impede a inicialização em produção se o segredo tiver menos de 32 caracteres ou for o valor de exemplo, que contém 'troque'. O segredo de desenvolvimento cai exatamente nesse filtro.

- ✔ **Cookie de sessão com as marcações corretas**
  AuthController::sessionCookie (235-249): HttpOnly, SameSite=Lax, path=/, Secure automático em produção e validade de 12 horas.

- ✔ **Segredos fora do versionamento**
  O .gitignore cobre .env*, com exceção dos exemplos. Os 69 commits do histórico não contêm .env nem chave de provedor — varredura pelos padrões sk-, AKIA, ghp_ e BEGIN PRIVATE KEY.

- ✔ **Renderização do conteúdo da aula sem HTML**
  src/components/student/LessonContent.tsx monta nós React. Há teste garantindo que '<script>alert(1)</script>' escrito no material apareça como texto: LessonContent.test.tsx, caso 'aplica negrito sem injetar HTML'. Os únicos dangerouslySetInnerHTML do projeto (App.tsx:611, :641 e :658) recebem CSS estático, sem interpolação de dado.
- ✔ **Fonte de vídeo restrita por lista branca**
  src/utils/videoSource.ts aceita apenas provedores conhecidos e caminhos /uploads/, rejeitando travessia de diretório. É o precedente que a correção do XSS-01 pode reaproveitar.
- ✔ **Upload com validação de conteúdo e proteção contra travessia**
  UploadService valida os bytes iniciais do arquivo, gera nome nunca reaproveitado (carimbo de tempo mais 8 bytes aleatórios) e bloqueia '/', '\\' e '..' na resolução. O download privado força Content-Disposition attachment e as respostas levam X-Content-Type-Options: nosniff.
- ✔ **Escrita privilegiada com papel verificado no servidor**
  Confirmado por php artisan route:list -v: RequireRole:admin em system-settings (PUT), site-content (PUT), security-logs (DELETE), document-templates (PUT e preview), export, auth/users/{id} (DELETE) e auth/users/{id}/status (PUT); RequireRole:instructor,admin nas alterações de curso, quiz, exercício e webinar. A concessão de matrícula múltipla e a reatribuição de autoria de curso são restritas ao admin dentro do service, não apenas na rota.
- ✔ **Posse verificada nas alterações de curso**
  CourseService::assertCourseOwnership é chamado em updateCourse (linha 100) e em deleteCourse (linha 142): o instrutor só altera curso próprio.
- ✔ **Nenhum SQL concatenado**
  Todo acesso a dados passa por Eloquent ou pelo query builder, com parâmetros vinculados. A varredura não encontrou DB::raw nem interpolação de entrada em SQL.
- ✔ **CORS sem origem liberada**
  Não existe config/cors.php, então o padrão do Laravel não libera origem externa. O frontend consome a API pela mesma origem, via proxy do Vite em desenvolvimento e via Nginx em produção.

Pontos fracos — os riscos centrais

Conteudo sem tranca na porta da frente

O isolamento por matricula existe e funciona no chat, nas mensagens diretas e nos certificados, mas nao no proprio material de estudo: GET /api/courses e publico e devolve as aulas inteiras. O controle certo esta implementado no lugar errado.

Segredo protegido por condicional de build

As senhas administrativas moram no codigo e sao "protegidas" por import.meta.env.DEV. Uma variavel de ambiente derrubou essa protecao inteira. Condicional de build nao e controle de acesso.

Escopo do instrutor aplicado de forma desigual

O mesmo projeto que limita o instrutor em DMs e certificados deixa passar forum, arquivos privados e solicitacoes academicas. A regra existe; falta uniformidade.

URL tratada como texto

Campos de URL sao validados apenas por tamanho e caem direto em href. O projeto ja tem o padrao correto em videoSource.ts — falta aplica-lo aos demais campos.

Achados detalhados

1. Banco sem tranca (isolamento)

ALTA **ISO-01 — Conteúdo integral das aulas exposto sem autenticação**

Local: `backend-laravel/routes/api.php:71 (GET /api/courses)`

```
Route::get('/courses', [CourseController::class, 'index']);
// nenhum middleware de autenticacao - apenas FeatureGate:catalogoCursos
```

Por que é explorável: A rota é pública e devolve o curso completo: cada aula com o campo 'content' (todo o material de estudo), 'videoUrl', a lista 'documents' (título, tipo e URL de cada anexo) e 'liveSessions' com o 'meetingLink'. Confirmado em execução: um 'curl -s /api/courses' retorna os 4 cursos com o texto das aulas, as URLs das apostilas e os links do Google Meet. Todo o isolamento por matrícula que o projeto aplica ao chat, às mensagens diretas e aos certificados simplesmente não existe para o material didático, que é o ativo central da plataforma. Além do vazamento do conteúdo, os meetingLink permitem que qualquer pessoa entre nas aulas ao vivo.

Impacto: Qualquer pessoa, sem conta e sem matrícula, baixa a íntegra do material de todos os cursos e obtém os links das aulas ao vivo.

Correção sugerida: Manter público apenas o resumo do catálogo (título, descrição, categoria, carga horária, capa) e exigir autenticação mais CourseAccess para content, videoUrl, documents e meetingLink. A alteração de menor impacto é uma projeção pública, com lista branca de campos, aplicada em CourseService::index quando o solicitante for anônimo.

MÉDIA **ISO-02 — Fórum de todos os cursos legível por qualquer usuário autenticado**

Local: `backend-laravel/app/Services/LearningService.php:166-170`

```
public function listForumMessages(): array
{
    return ForumMessage::query()->get()->map->toArray()->all();
}
```

Por que é explorável: O método não recebe nem usa o solicitante: devolve todas as mensagens de todos os cursos. O serviço irmão MessagingService::listChatMessages (linhas 31 a 52) faz exatamente o oposto e filtra por CourseAccess::canAccess, o que demonstra que o escopo por turma é a regra do projeto — aqui ela está ausente. Um aluno matriculado em um único curso lê a discussão de todas as turmas da escola.

Impacto: Vazamento das discussões entre turmas, incluindo dúvidas e informações que os alunos expõem no fórum de cursos alheios.

Correção sugerida: Filtrar por CourseAccess::accessibleCourseIds($requester), sem filtro para o perfil admin, espelhando o que listChatMessages já faz.

MÉDIA **ISO-03 — Escrita no fórum de curso alheio: o courseId vem do corpo, sem checagem**

Local: `backend-laravel/app/Services/LearningService.php:176-193`

```
return ForumMessage::query()->create([
    'courseId' => $input['courseId'], // aceito sem verificar acesso
    'senderUserId' => $requester['sub'],
    ...
```

Por que é explorável: O courseId é usado diretamente do corpo da requisição. Não há chamada a CourseAccess::canAccess como existe em MessagingService::createChatMessage (linhas 61 a 64), que rejeita com 403 e a mensagem 'Você não participa desta turma'. Assim, qualquer usuário autenticado publica em qualquer curso.

Impacto: Inserção de mensagens em turmas das quais o autor não participa, exibidas com o nome e o papel reais dele — vetor de spam e de engenharia social contra alunos de outros cursos.

Correção sugerida: Chamar CourseAccess::canAccess($requester, $input['courseId']) e lançar 403 quando o acesso não existir.

MÉDIA **ISO-04 — Instrutor lê as justificativas acadêmicas de toda a escola**

Local: `backend-laravel/app/Services/RequestService.php:22-30`

```
$q = AcademicRequest::query();
if ($requester['role'] === 'student') {
    Identity::applyOwnRows($q, $requester);
}
// instrutor e admin: nenhum filtro
```

Por que é explorável: Somente o aluno é limitado aos próprios registros. O instrutor recebe todas as solicitações da instituição, e cada uma contém uma descrição livre escrita pelo aluno, com motivos de saúde, familiares ou pessoais. Nos demais serviços o mesmo código aplica InstructorScope — veja CertificateService:28-45 e MessagingService:88-99 —, o que torna esta ausência inconsistente. O comentário da classe declara a escolha ('secretaria centralizada'), portanto pode ser intencional; ainda assim, expõe dado pessoal sensível a quem não leciona para aquele aluno.

Impacto: Exposição de dados pessoais sensíveis de alunos a instrutores sem qualquer vínculo com eles.

Correção sugerida: Se a centralização for mesmo desejada, restringir a leitura ampla ao perfil admin e aplicar InstructorScope::studentIds ao instrutor. Caso contrário, registrar a decisão em ADR e passar a auditar esse acesso.

MÉDIA **ISO-05 — Gabarito de todas as avaliações entregue a qualquer usuário autenticado**

Local: `backend-laravel/app/Services/LearningService.php:32-36`

```
public function listQuizzes(): array
{
    return Quiz::query()->with('questions')->get()->map->toArray()->all();
}
```

Por que é explorável: O carregamento de 'questions' inclui o campo correctOptionIndex. Qualquer aluno autenticado obtém a resposta correta de todas as avaliações de todos os cursos com um GET em /api/quizzes. A nota em si não pode ser forjada, porque submitQuiz recalcula no servidor — esse é um ponto forte do projeto —, portanto o impacto recai sobre a integridade acadêmica, não sobre elevação de privilégio.

Impacto: Fraude em avaliações: o gabarito fica disponível antes da tentativa.

Correção sugerida: Omitir correctOptionIndex na listagem e devolvê-lo apenas na correção, dentro da resposta de submitQuiz, ou expor o gabarito somente aos perfis instrutor e admin.

BAIXA **ISO-06 — Enunciados de exercícios de todos os cursos sem escopo**

Local: `backend-laravel/app/Services/LearningService.php:231-234`

```
public function listExercises(): array
{
    return PracticalExercise::query()->get()->map->toArray()->all();
}
```

Por que é explorável: Não há filtro por curso nem por acesso. É a mesma classe de problema do ISO-02, com conteúdo menos sensível: o enunciado, não a resposta. As submissões, por sua vez, são filtradas corretamente em listExerciseSubmissions.

Impacto: Vazamento de enunciados entre turmas.

Correção sugerida: Filtrar por CourseAccess::accessibleCourseIds.

2. Permissão definida no navegador

MÉDIA **PRIV-01 — Emissão de certificado sem verificação de papel no servidor**

Local: `backend-laravel/routes/api.php:118-119 (POST /api/certificates)` e `app/Services/CertificateService.php:84-130`

```
Route::post('/certificates', [CertificateController::class, 'store'])
    ->middleware(['jwt', 'active']); // sem role:
// no service:
$userId = Identity::resolveActorUserId($requester, $input['userId'] ?? null);
```

Por que é explorável: A rota aceita qualquer usuário autenticado. Para o aluno o risco é contido: resolveActorUserId força o próprio identificador e a frequência mínima é recalculada no servidor (linhas 106 a 113), e ambas são proteções reais. O problema está no instrutor: ele informa qualquer userId e não existe verificação de InstructorScope sobre o curso, ao contrário do que o próprio projeto faz no download do PDF (CertificatePdfService.php:41-44). Um instrutor emite, portanto, certificado para aluno de curso que não leciona.

Impacto: Emissão de documento acadêmico por quem não tem relação com o curso, com registro de autoria indevida.

Correção sugerida: Acrescentar role:instructor,admin na rota e validar InstructorScope::courseIds no service quando o solicitante for instrutor e o userId pertencer a outra pessoa.

BAIXA **PRIV-02 — Configurações do sistema legíveis publicamente e gravadas sem esquema**

Local: `backend-laravel/routes/api.php (GET /api/system-settings, sem middleware)` e `app/Services/SettingsService.php:16-40`

```
public function update(array $updates): array
{
    $merged = array_merge($row->data ?? [], $updates); // aceita chaves arbitrarias
```

Por que é explorável: A leitura é pública — confirmado por curl, que devolve autoCertify, openEnrollment, allowGlobalChat, liveClassRecording, allowDirectMessages e autoArchiveDuration — e a escrita, restrita ao admin, faz merge de qualquer chave enviada, sem esquema algum. Hoje o conteúdo é apenas operacional; o risco é futuro e silencioso, porque qualquer campo sensível que um administrador passe a guardar ali nasce público.

Impacto: Divulgação da configuração operacional e risco de exposição de dado sensível em alterações futuras.

Correção sugerida: Exigir autenticação na leitura, ou devolver uma lista branca de chaves públicas, e validar quais chaves são aceitas na escrita.

3. IDOR

MÉDIA **IDOR-01 — Qualquer instrutor baixa arquivo privado de qualquer aluno**

Local: `backend-laravel/app/Services/UploadService.php:95-99 (GET /api/files/{id})`

```
$isOwner = $record->ownerUserId === $requester['sub'];
$isStaff = in_array($requester['role'] ?? null, ['instructor', 'admin'], true);
if ($record->visibility === 'private' && ! $isOwner && ! $isStaff) {
    throw ApiException::forbidden('Voce nao tem permissao para acessar este arquivo.');
}
```

Por que é explorável: A condição 'isStaff' é concedida a qualquer instrutor, sem escopo de curso ou de aluno. O identificador do arquivo é obtido a partir das submissões, que o instrutor já lista. Com isso ele baixa a entrega privada de um aluno de turma alheia — enquanto o mesmo código aplica InstructorScope para certificados (CertificatePdfService.php:41-44) e para mensagens diretas (MessagingService.php:91-96).

Impacto: Leitura de documentos privados — entregas, atestados, comprovantes — de alunos fora do escopo do instrutor.

Correção sugerida: Para o perfil instrutor, exigir que o dono do arquivo esteja em InstructorScope::studentIds($requester), mantendo acesso amplo apenas para o admin.

MÉDIA **IDOR-02 — Aprovação e rejeição de solicitação acadêmica sem escopo**

Local: `backend-laravel/app/Services/RequestService.php:55-68 (PUT /api/academic-requests/{id})`

```
public function updateAcademicRequestStatus(string $id, string $status): array
{
    $req = AcademicRequest::query()->find($id);
    // nenhuma checagem de escopo: o requester nem e recebido
    $req->status = $status;
```

Por que é explorável: O método sequer recebe o solicitante, então não há como checar escopo. Somado ao ISO-04, que entrega ao instrutor a lista completa de identificadores, qualquer instrutor decide o pedido de qualquer aluno da instituição. A rota exige role:instructor,admin, portanto não é acessível a alunos.

Impacto: Decisão acadêmica — o deferimento de uma justificativa — tomada por quem não leciona para o aluno, sem qualquer rastro de escopo.

Correção sugerida: Receber o solicitante e, para o perfil instrutor, exigir que o userId da solicitação esteja em InstructorScope::studentIds.

4. Chaves expostas (hardcode)

CRÍTICA **SEC-01 — NODE_ENV=development no .env faz o build de produção publicar as senhas de admin**

Local: `.env:12 — consequência observada em dist/assets/index-*.js, a partir de src/App.tsx:2930-2941 e src/components/ProfileView.tsx:26-30 e 1563-1583`

```
# .env (raiz do projeto, lido pelo Vite em qualquer modo)
NODE_ENV="development"

// src/App.tsx:2930 - bloco que DEVERIA existir so em desenvolvimento
{import.meta.env.DEV && (
  ... "Aluno: 1234", "Gestao: 5678", "Admin Superior: 9999" ...
)}
```

Por que é explorável: O Vite lê o .env em todos os modos e aplica NODE_ENV ao processo. Com isso, 'npm run build' gera um pacote de desenvolvimento: import.meta.env.DEV vale verdadeiro e todo bloco protegido por essa condição vai ativo para produção. Verificado no artefato gerado: grep por 'Dica para Avaliação do Fluxo'

no bundle retorna uma ocorrência, sem o guard '!1&&' que a eliminaria, e o grep por pin:"[0-9]*" retorna pin:"1234", pin:"5678" e pin:"9999". Rodar 'npx vite build --mode production' não corrige, porque o .env prevalece. Na prática, a tela de login de produção exibe as senhas de Aluno, Gestão e Admin Superior, e o painel de troca rápida de perfil (ProfileView:1563) oferece entrar como Admin com um clique. O pacote ainda carrega jsxDEV com caminhos absolutos do disco do desenvolvedor, em 19 ocorrências, e executa o React em modo de desenvolvimento.

Impacto: Tomada de conta administrativa por qualquer visitante da página de login: a credencial está impressa na tela e o atalho de entrada como Admin está renderizado ao lado.

Correção sugerida: Remover NODE_ENV do .env, já que o Vite define o modo sozinho — desenvolvimento no 'vite' e produção no 'vite build'. Depois do build, confirmar que 'Dica para Avaliação' e pin:"9999" desapareceram de dist/. E, principalmente, não usar import.meta.env.DEV para proteger segredo: credencial não deveria existir no código (ver SEC-02).

ALTA **SEC-02 — Senhas reais de Admin e Gestor embutidas no código-fonte do frontend**

Local: `src/components/ProfileView.tsx:26-30`

```
const DEMO_PROFILES = [
  { name: 'Joao Silva', pin: '1234', label: 'Aluno' },
  { name: 'Gestor de Conteudos', pin: '5678', label: 'Gestor' },
  { name: 'Admin Superior', pin: '9999', label: 'Admin' },
] as const;
```

Por que é explorável: São as credenciais em uso das contas administrativas, escritas no código-fonte. O array está no escopo do módulo e sobrevive no pacote entregue ao navegador — confirmado, pin:"9999" aparece em dist/assets/index-*.js — independentemente do guard import.meta.env.DEV, que protege apenas a interface. Como a API aceita login por nome (AuthService), o par nome mais PIN basta para autenticar.

Impacto: Credencial administrativa distribuída a todo visitante que abrir o JavaScript da aplicação.

Correção sugerida: Remover o array do código, trocar as senhas de admin e gestor por valores fortes e, se o atalho de troca de perfil for necessário em desenvolvimento, ler os PINs de variáveis VITE_* vindas de um .env.local não versionado, que ficam vazias em produção.

MÉDIA **SEC-03 — Senha padrão '1234' na criação de aluno, exibida na própria interface**

Local: `src/components/AdminDashboard.tsx:602, :263 e :1329`

```
const pass = newStudentPassword.trim() || '1234'; // :602
const activePass = st.password || localStorage.getItem(...) || '1234'; // :263
<span ...>5678 ou 1234</span> // :1329
```

Por que é explorável: Quando o administrador cadastra um aluno sem informar senha, a conta nasce com '1234'. O valor ainda é exibido na tela de gestão como dica. A política de senha forte (StrongPasswordRule) é aplicada no registro público e na troca de senha, mas este caminho administrativo entrega um padrão adivinhável e publicamente conhecido.

Impacto: Contas de alunos recém-criadas acessíveis por tentativa trivial.

Correção sugerida: Gerar senha aleatória quando o campo vier vazio — o LMSContext.tsx:1908 já faz isso em outro fluxo e documenta o motivo —, exibi-la uma única vez ao administrador e exigir troca no primeiro acesso. Remover também o texto que mostra os padrões na interface.

5. Entradas sem tratamento (XSS)

ALTA **XSS-01 — URL de anexo, biblioteca e aula ao vivo sem validação de esquema em href**

Local: `Backend: backend-laravel/app/Http/Controllers/CourseController.php:96 e :104; app/Http/Controllers/LibraryController.php:41. Frontend: src/components/StudentDashboard.tsx:1049, :1711 e :1945; src/components/student/StudentLibraryPanel.tsx:41; src/components/LiveClassroom.tsx:530; src/components/InstructorDashboard.tsx:623 e :2812`

```
// a validacao aceita qualquer string
'lessons.*.documents.*.url' => ['required_with:...', 'string', 'min:1', 'max:2000'],
'liveSessions.*.meetingLink' => ['required_with:...', 'string', 'min:1', 'max:2000'],
'url' => ['required', 'string', 'max:2000'], // LibraryController

// e o valor vai direto para o atributo
<a href={doc.url} target="_blank" ...>
```

Por que é explorável: Nenhuma das três validações restringe o esquema da URL, e o valor é usado diretamente no atributo href. Um instrutor ou administrador salva 'javascript:...' como URL de documento, de item da biblioteca ou de link da aula ao vivo; quando o aluno clica, o script executa na origem da aplicação e dentro da sessão dele. O cookie ava_session é HttpOnly, e portanto não é legível pelo script, mas o navegador o envia automaticamente — logo o código injetado chama a API como se fosse o aluno, podendo trocar a senha, enviar mensagens ou cancelar a matrícula. Não existe biblioteca de sanitização no projeto, conforme verificado no package.json.

Impacto: XSS armazenado com escalada entre perfis: uma conta de instrutor ou administrador comprometida executa ações em nome de qualquer aluno que abra o material.

Correção sugerida: Validar no backend que a URL comece por http:// ou https:// — regra 'url' do Laravel ou expressão regular de esquema — e, no frontend, passar todo href por uma única função que rejeite esquemas fora da lista permitida. O projeto já tem esse padrão em src/utils/videoSource.ts, que restringe as fontes de vídeo aceitas.

BAIXA **XSS-02 — Anotações do aluno manipulam HTML cru, sem sanitização**

Local: `src/components/StudentDashboard.tsx:316, :340 e :351`

```
noteEditorRef.current.innerHTML = savedNotes[activeLesson.id] || ''; // :316
const html = noteEditorRef.current.innerHTML; // :340 e :351
const htmlDoc = `<!DOCTYPE html>... ${html}</body></html>`; // exportacao
```

Por que é explorável: O editor de anotações é contentEditable e o conteúdo trafega como HTML cru. A origem do dado é o localStorage do próprio usuário, na chave ava_student_lesson_notes, e nunca outro usuário ou o servidor — trata-se, portanto, de self-XSS: para explorar, a vítima precisa injetar em si mesma. O ponto que merece atenção é a exportação: o HTML vai para um arquivo .html baixado, que abre como página local com script ativo caso o conteúdo tenha sido colado de uma fonte hostil.

Impacto: Baixo: a execução fica restrita à própria sessão. O arquivo exportado pode carregar script colado de terceiros.

Correção sugerida: Sanitizar na leitura e na exportação, com lista branca de tags de formatação, ou substituir o contentEditable por marcação simples, como já é feito no conteúdo da aula (src/utils/lessonContent.ts).

Condições de explorabilidade

- ISO-01, SEC-01 e SEC-02 são exploráveis por qualquer visitante anônimo, sem pré-condição alguma.
- ISO-02, ISO-03, ISO-05, ISO-06 e PRIV-01 exigem apenas uma conta válida de aluno.
- IDOR-01, IDOR-02 e ISO-04 exigem conta de instrutor: não são alcançáveis por um aluno.
- XSS-01 exige que quem planta a URL tenha conta de instrutor ou de administrador; a vítima é o aluno.

- ISO-01, ISO-02, ISO-03, ISO-05 e ISO-06 dependem de as feature flags catalogoCursos, forum, quizSimples e atividadesPraticasAvancadas estarem ligadas em config/features.php. Com a flag desligada, a rota responde 404 FEATURE_DISABLED.
- SEC-01 depende de o artefato ser gerado com o .env do repositório presente, que é justamente o fluxo documentado em DEPLOY_LARAVEL.md.
- XSS-02 é self-XSS: não há caminho de um usuário para outro.

Recomendações priorizadas

Prio.	Sev.	Ação
P1	CRÍTICA	Remover NODE_ENV do .env e confirmar, no artefato gerado, que a dica de PINs e o array DEMO_PROFILES desapareceram do pacote (SEC-01).
P1	ALTA	Trocar as senhas de Admin Superior e Gestor de Conteúdos e apagar os PINs do código-fonte (SEC-02).
P1	ALTA	Fechar o GET /api/courses: manter público apenas o resumo do catálogo e exigir autenticação mais CourseAccess para content, videoUrl, documents e meetingLink (ISO-01).
P2	ALTA	Validar o esquema http/https nas URLs de documento, biblioteca e meetingLink, no backend e no href do frontend (XSS-01).
P2	MÉDIA	Aplicar escopo de curso e de aluno no fórum (leitura e escrita), no download de arquivo privado pelo instrutor e na decisão de solicitação acadêmica (ISO-02, ISO-03, IDOR-01 e IDOR-02).
P2	MÉDIA	Parar de entregar correctOptionIndex na listagem de avaliações (ISO-05).
P3	MÉDIA	Substituir a senha padrão '1234' por senha aleatória de uso único, com troca obrigatória, e remover os padrões exibidos na interface (SEC-03).
P3	MÉDIA	Definir se a leitura ampla de justificativas acadêmicas pelo instrutor é intencional; em caso afirmativo, registrar em ADR e auditar o acesso (ISO-04).
P3	BAIXA	Exigir autenticação, ou lista branca de chaves, no GET /api/system-settings e validar as chaves aceitas na escrita (PRIV-02).
P3	BAIXA	Sanitizar o HTML das anotações na leitura e na exportação (XSS-02).

Issues para o GitHub

Cada bloco abaixo é o texto completo de uma issue, pronto para copiar e colar. Achados do mesmo tema foram agrupados para não gerar ruído no quadro.

--- ISSUE 1 ---

```
## [Seguranca] Build de producao sai em modo dev e publica as senhas de Admin e Gestor

**Labels:** security, critica

### Problema
O Vite lê o .env em todos os modos e aplica NODE_ENV ao processo. Com isso, 'npm run build' gera um
pacote de desenvolvimento: import.meta.env.DEV vale verdadeiro e todo bloco protegido por essa
condição vai ativo para produção. Verificado no artefato gerado: grep por 'Dica para Avaliação do
Fluxo' no bundle retorna uma ocorrência, sem o guard '!1&&' que a eliminaria, e o grep por
pin:"[0-9]*" retorna pin:"1234", pin:"5678" e pin:"9999". Rodar 'npx vite build --mode production'
não corrige, porque o .env prevalece. Na prática, a tela de login de produção exibe as senhas de
Aluno, Gestão e Admin Superior, e o painel de troca rápida de perfil (ProfileView:1563) oferece
entrar como Admin com um clique. O pacote ainda carrega jsxDEV com caminhos absolutos do disco do
desenvolvedor, em 19 ocorrências, e executa o React em modo de desenvolvimento.

### Evidencia
**Arquivo:** `.env:12 — consequência observada em dist/assets/index-*.js, a partir de
src/App.tsx:2930-2941 e src/components/ProfileView.tsx:26-30 e 1563-1583`

```
# .env (raiz do projeto, lido pelo Vite em qualquer modo)
NODE_ENV="development"

// src/App.tsx:2930 - bloco que DEVERIA existir so em desenvolvimento
{import.meta.env.DEV && (
  ... "Aluno: 1234", "Gestao: 5678", "Admin Superior: 9999" ...
)}
```

O Vite lê o .env em todos os modos e aplica NODE_ENV ao processo. Com isso, 'npm run build' gera um
pacote de desenvolvimento: import.meta.env.DEV vale verdadeiro e todo bloco protegido por essa
condição vai ativo para produção. Verificado no artefato gerado: grep por 'Dica para Avaliação do
Fluxo' no bundle retorna uma ocorrência, sem o guard '!1&&' que a eliminaria, e o grep por
pin:"[0-9]*" retorna pin:"1234", pin:"5678" e pin:"9999". Rodar 'npx vite build --mode production'
não corrige, porque o .env prevalece. Na prática, a tela de login de produção exibe as senhas de
Aluno, Gestão e Admin Superior, e o painel de troca rápida de perfil (ProfileView:1563) oferece
entrar como Admin com um clique. O pacote ainda carrega jsxDEV com caminhos absolutos do disco do
desenvolvedor, em 19 ocorrências, e executa o React em modo de desenvolvimento.

Reproducao:
```bash
npm run build
grep -c 'Dica para Avaliacao do Fluxo' dist/assets/index-*.js # 1 = dica de PIN no bundle
grep -o 'pin:"[0-9]*"' dist/assets/index-*.js # pin:"1234" pin:"5678" pin:"9999"
grep -c jsxDEV dist/assets/index-*.js # 19 = React em modo dev
```

Impacto
Tomada de conta administrativa por qualquer visitante da página de login: a credencial está impressa
na tela e o atalho de entrada como Admin está renderizado ao lado.
```

```
### Correcao sugerida
Remover NODE_ENV do .env, já que o Vite define o modo sozinho — desenvolvimento no 'vite' e produção
no 'vite build'. Depois do build, confirmar que 'Dica para Avaliação' e pin:"9999" desapareceram de
dist/. E, principalmente, não usar import.meta.env.DEV para proteger segredo: credencial não deveria
existir no código (ver SEC-02).

### Criterios de aceite
- [ ] `NODE_ENV` removido de `.env` (e documentado em `.env.example` que o Vite define o modo)
- [ ] Apos `npm run build`, `grep -c 'Dica para Avaliacao' dist/assets/index-*.js` retorna 0
- [ ] Apos `npm run build`, `grep -c jsxDEV dist/assets/index-*.js` retorna 0
- [ ] Nenhum caminho absoluto do disco do desenvolvedor aparece no bundle
- [ ] Teste ou script de CI que falhe se um marcador de dev voltar ao artefato
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
## [Seguranca] Credenciais administrativas e senha padrao embutidas no codigo

**Labels:** security, alta

### Problema
São as credenciais em uso das contas administrativas, escritas no código-fonte. O array está no
escopo do módulo e sobrevive no pacote entregue ao navegador — confirmado, pin:"9999" aparece em
dist/assets/index-*.js — independentemente do guard import.meta.env.DEV, que protege apenas a
interface. Como a API aceita login por nome (AuthService), o par nome mais PIN basta para
autenticar.

Relacionado (mesmo tema, severidade media): Senha padrão '1234' na criação de aluno, exibida na
própria interface. Quando o administrador cadastra um aluno sem informar senha, a conta nasce com
'1234'. O valor ainda é exibido na tela de gestão como dica. A política de senha forte
(StrongPasswordRule) é aplicada no registro público e na troca de senha, mas este caminho
administrativo entrega um padrão adivinhável e publicamente conhecido.

### Evidencia
**Arquivo:** `src/components/ProfileView.tsx:26-30`

```
const DEMO_PROFILES = [
  { name: 'Joao Silva', pin: '1234', label: 'Aluno' },
  { name: 'Gestor de Conteudos', pin: '5678', label: 'Gestor' },
  { name: 'Admin Superior', pin: '9999', label: 'Admin' },
] as const;
```

São as credenciais em uso das contas administrativas, escritas no código-fonte. O array está no
escopo do módulo e sobrevive no pacote entregue ao navegador — confirmado, pin:"9999" aparece em
dist/assets/index-*.js — independentemente do guard import.meta.env.DEV, que protege apenas a
interface. Como a API aceita login por nome (AuthService), o par nome mais PIN basta para
autenticar.

Arquivo: `src/components/AdminDashboard.tsx:602, :263 e :1329`

```
const pass = newStudentPassword.trim() || '1234'; // :602
const activePass = st.password || localStorage.getItem(...) || '1234'; // :263
<span ...>5678 ou 1234 // :1329
```
```

```
Quando o administrador cadastra um aluno sem informar senha, a conta nasce com '1234'. O valor ainda é exibido na tela de gestão como dica. A política de senha forte (StrongPasswordRule) é aplicada no registro público e na troca de senha, mas este caminho administrativo entrega um padrão adivinhável e publicamente conhecido.

### Impacto
Credencial administrativa distribuída a todo visitante que abrir o JavaScript da aplicação.

### Correcao sugerida
Remover o array do código, trocar as senhas de admin e gestor por valores fortes e, se o atalho de troca de perfil for necessário em desenvolvimento, ler os PINs de variáveis VITE_* vindas de um .env.local não versionado, que ficam vazias em produção.

Para a senha padrao: Gerar senha aleatória quando o campo vier vazio — o LMSContext.tsx:1908 já faz isso em outro fluxo e documenta o motivo —, exibi-la uma única vez ao administrador e exigir troca no primeiro acesso. Remover também o texto que mostra os padrões na interface.

### Criterios de aceite
- [ ] Nenhum PIN literal em `src/` (`grep -rn "pin: '" src/` sem resultado)
- [ ] Senhas de Admin Superior e Gestor de Conteudos trocadas por valores fortes
- [ ] Cadastro administrativo de aluno sem senha gera valor aleatorio de uso unico
- [ ] Nenhum texto de interface exibe senha padrao
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```
## [Seguranca] GET /api/courses expoe o conteudo integral das aulas sem autenticacao

**Labels:** security, alta

### Problema
A rota é pública e devolve o curso completo: cada aula com o campo 'content' (todo o material de estudo), 'videoUrl', a lista 'documents' (título, tipo e URL de cada anexo) e 'liveSessions' com o 'meetingLink'. Confirmado em execução: um 'curl -s /api/courses' retorna os 4 cursos com o texto das aulas, as URLs das apostilas e os links do Google Meet. Todo o isolamento por matrícula que o projeto aplica ao chat, às mensagens diretas e aos certificados simplesmente não existe para o material didático, que é o ativo central da plataforma. Além do vazamento do conteúdo, os meetingLink permitem que qualquer pessoa entre nas aulas ao vivo.

### Evidencia
**Arquivo:** `backend-laravel/routes/api.php:71 (GET /api/courses)`

```
Route::get('/courses', [CourseController::class, 'index']);
// nenhum middleware de autenticacao - apenas FeatureGate:catalogoCursos
```

A rota é pública e devolve o curso completo: cada aula com o campo 'content' (todo o material de estudo), 'videoUrl', a lista 'documents' (título, tipo e URL de cada anexo) e 'liveSessions' com o 'meetingLink'. Confirmado em execução: um 'curl -s /api/courses' retorna os 4 cursos com o texto das aulas, as URLs das apostilas e os links do Google Meet. Todo o isolamento por matrícula que o projeto aplica ao chat, às mensagens diretas e aos certificados simplesmente não existe para o material didático, que é o ativo central da plataforma. Além do vazamento do conteúdo, os meetingLink permitem que qualquer pessoa entre nas aulas ao vivo.
```

Reproducao:

```bash
curl -s http://localhost:8000/api/courses | head -c 800 # content, documents e meetingLink
```

Impacto

Qualquer pessoa, sem conta e sem matrícula, baixa a íntegra do material de todos os cursos e obtém os links das aulas ao vivo.

Correcao sugerida

Manter público apenas o resumo do catálogo (título, descrição, categoria, carga horária, capa) e exigir autenticação mais CourseAccess para content, videoUrl, documents e meetingLink. A alteração de menor impacto é uma projeção pública, com lista branca de campos, aplicada em CourseService::index quando o solicitante for anônimo.

Criterios de aceite

- [ ] Requisicao anonima a `/api/courses` nao devolve `content`, `videoUrl`, `documents` nem `liveSessions[].meetingLink`
- [ ] Aluno matriculado continua recebendo o curso completo
- [ ] Aluno nao matriculado nao recebe o conteudo das aulas
- [ ] Teste de feature cobrindo os tres casos acima

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Seguranca] URLs de anexo, biblioteca e aula ao vivo aceitam esquema javascript:

Labels: security, alta

Problema

Nenhuma das três validações restringe o esquema da URL, e o valor é usado diretamente no atributo href. Um instrutor ou administrador salva 'javascript:...' como URL de documento, de item da biblioteca ou de link da aula ao vivo; quando o aluno clica, o script executa na origem da aplicação e dentro da sessão dele. O cookie ava_session é HttpOnly, e portanto não é legível pelo script, mas o navegador o envia automaticamente — logo o código injetado chama a API como se fosse o aluno, podendo trocar a senha, enviar mensagens ou cancelar a matrícula. Não existe biblioteca de sanitização no projeto, conforme verificado no package.json.

Evidencia

Arquivo: `Backend: backend-laravel/app/Http/Controllers/CourseController.php:96 e :104; app/Http/Controllers/LibraryController.php:41. Frontend: src/components/StudentDashboard.tsx:1049, :1711 e :1945; src/components/student/StudentLibraryPanel.tsx:41; src/components/LiveClassroom.tsx:530; src/components/InstructorDashboard.tsx:623 e :2812`

```
// a validacao aceita qualquer string
'lessons.*.documents.*.url' => ['required_with:...', 'string', 'min:1', 'max:2000'],
'liveSessions.*.meetingLink' => ['required_with:...', 'string', 'min:1', 'max:2000'],
'url' => ['required', 'string', 'max:2000'], // LibraryController

// e o valor vai direto para o atributo
<a href={doc.url} target="_blank" ...>
```

Nenhuma das três validações restringe o esquema da URL, e o valor é usado diretamente no atributo href. Um instrutor ou administrador salva 'javascript:...' como URL de documento, de item da biblioteca ou de link da aula ao vivo; quando o aluno clica, o script executa na origem da aplicação e dentro da sessão dele. O cookie ava_session é HttpOnly, e portanto não é legível pelo script, mas o navegador o envia automaticamente — logo o código injetado chama a API como se fosse o aluno, podendo trocar a senha, enviar mensagens ou cancelar a matrícula. Não existe biblioteca de sanitização no projeto, conforme verificado no package.json.

Impacto

XSS armazenado com escalada entre perfis: uma conta de instrutor ou administrador comprometida executa ações em nome de qualquer aluno que abra o material.

Correcao sugerida

Validar no backend que a URL comece por http:// ou https:// — regra 'url' do Laravel ou expressão regular de esquema — e, no frontend, passar todo href por uma única função que rejeite esquemas fora da lista permitida. O projeto já tem esse padrão em src/utils/videoSource.ts, que restringe as fontes de vídeo aceitas.

Criterios de aceite

- [ ] Backend rejeita com 400 uma URL que nao comece por `http://` ou `https://` em documento, biblioteca e meetingLink
- [ ] Frontend nao renderiza href com esquema fora da whitelist (funcao unica, testada)
- [ ] Teste com payload `javascript:alert(1)` cobrindo os dois lados

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Seguranca] Escopo de curso/aluno ausente em forum, arquivos privados e solicitacoes

Labels: security, media

Problema

Cinco pontos aplicam o filtro de escopo de forma inconsistente com o resto do projeto, que ja possui os helpers corretos (`CourseAccess`, `InstructorScope`, `Identity`).

- **ISO-02** — Fórum de todos os cursos legível por qualquer usuário autenticado
- **ISO-03** — Escrita no fórum de curso alheio: o courseId vem do corpo, sem checagem
- **IDOR-01** — Qualquer instrutor baixa arquivo privado de qualquer aluno
- **IDOR-02** — Aprovação e rejeição de solicitação acadêmica sem escopo
- **ISO-04** — Instrutor lê as justificativas acadêmicas de toda a escola

Evidencia

Arquivo: `backend-laravel/app/Services/LearningService.php:166-170`

```
public function listForumMessages(): array
{
    return ForumMessage::query()->get()->map->toArray()->all();
}
```

O método não recebe nem usa o solicitante: devolve todas as mensagens de todos os cursos. O serviço irmão MessagingService::listChatMessages (linhas 31 a 52) faz exatamente o oposto e filtra por CourseAccess::canAccess, o que demonstra que o escopo por turma é a regra do projeto — aqui ela está ausente. Um aluno matriculado em um único curso lê a discussão de todas as turmas da escola.

Arquivo: `backend-laravel/app/Services/LearningService.php:176-193`

```
return ForumMessage::query()->create([
    'courseId' => $input['courseId'], // aceito sem verificar acesso
    'senderUserId' => $requester['sub'],
    ...
```

O courseId é usado diretamente do corpo da requisição. Não há chamada a CourseAccess::canAccess como existe em MessagingService::createChatMessage (linhas 61 a 64), que rejeita com 403 e a mensagem 'Você não participa desta turma'. Assim, qualquer usuário autenticado publica em qualquer curso.

Arquivo: `backend-laravel/app/Services/UploadService.php:95-99 (GET /api/files/{id})`

```
$isOwner = $record->ownerUserId === $requester['sub'];
$isStaff = in_array($requester['role'] ?? null, ['instructor', 'admin'], true);
if ($record->visibility === 'private' && ! $isOwner && ! $isStaff) {
    throw ApiException::forbidden('Voce nao tem permissao para acessar este arquivo.');
}
```

A condição 'isStaff' é concedida a qualquer instrutor, sem escopo de curso ou de aluno. O identificador do arquivo é obtido a partir das submissões, que o instrutor já lista. Com isso ele baixa a entrega privada de um aluno de turma alheia — enquanto o mesmo código aplica InstructorScope para certificados (CertificatePdfService.php:41-44) e para mensagens diretas (MessagingService.php:91-96).

Arquivo: `backend-laravel/app/Services/RequestService.php:55-68 (PUT /api/academic-requests/{id})`

```
public function updateAcademicRequestStatus(string $id, string $status): array
{
    $req = AcademicRequest::query()->find($id);
    // nenhuma checagem de escopo: o requester nem e recebido
    $req->status = $status;
```

O método sequer recebe o solicitante, então não há como checar escopo. Somado ao ISO-04, que entrega ao instrutor a lista completa de identificadores, qualquer instrutor decide o pedido de qualquer aluno da instituição. A rota exige role:instructor,admin, portanto não é acessível a alunos.

Arquivo: `backend-laravel/app/Services/RequestService.php:22-30`

```
$q = AcademicRequest::query();
if ($requester['role'] === 'student') {
    Identity::applyOwnRows($q, $requester);
}
// instrutor e admin: nenhum filtro
```

Somente o aluno é limitado aos próprios registros. O instrutor recebe todas as solicitações da instituição, e cada uma contém uma descrição livre escrita pelo aluno, com motivos de saúde, familiares ou pessoais. Nos demais serviços o mesmo código aplica InstructorScope — veja CertificateService:28-45 e MessagingService:88-99 —, o que torna esta ausência inconsistente. O comentário da classe declara a escolha ('secretaria centralizada'), portanto pode ser intencional; ainda assim, expõe dado pessoal sensível a quem não leciona para aquele aluno.

Impacto

Leitura e escrita entre turmas, download de documento privado de aluno fora do escopo do instrutor e decisao academica sem vinculo com o aluno.

Correcao sugerida

- **ISO-02**: Filtrar por CourseAccess::accessibleCourseIds($requester), sem filtro para o perfil admin, espelhando o que listChatMessages já faz.
- **ISO-03**: Chamar CourseAccess::canAccess($requester, $input['courseId']) e lançar 403 quando o acesso não existir.
- **IDOR-01**: Para o perfil instrutor, exigir que o dono do arquivo esteja em InstructorScope::studentIds($requester), mantendo acesso amplo apenas para o admin.
- **IDOR-02**: Receber o solicitante e, para o perfil instrutor, exigir que o userId da solicitação esteja em InstructorScope::studentIds.
- **ISO-04**: Se a centralização for mesmo desejada, restringir a leitura ampla ao perfil admin e aplicar InstructorScope::studentIds ao instrutor. Caso contrário, registrar a decisão em ADR e passar a auditar esse acesso.

Criterios de aceite

- [ ] Aluno so le e escreve no forum de cursos a que tem acesso
- [ ] Instrutor so baixa arquivo privado de aluno dentro do seu escopo
- [ ] Instrutor so decide solicitacao academica de aluno dentro do seu escopo
- [ ] Decisao sobre a leitura ampla de solicitacoes registrada em ADR
- [ ] Teste de feature para cada regra acima

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Seguranca] Listagem de quizzes entrega o gabarito a qualquer autenticado

Labels: security, media

Problema

O carregamento de 'questions' inclui o campo correctOptionIndex. Qualquer aluno autenticado obtém a resposta correta de todas as avaliações de todos os cursos com um GET em /api/quizzes. A nota em si não pode ser forjada, porque submitQuiz recalcula no servidor — esse é um ponto forte do projeto —, portanto o impacto recai sobre a integridade acadêmica, não sobre elevação de privilégio.

Relacionado: Enunciados de exercícios de todos os cursos sem escopo (ISO-06, severidade baixa).

Evidencia

Arquivo: `backend-laravel/app/Services/LearningService.php:32-36`

```
public function listQuizzes(): array
{
    return Quiz::query()->with('questions')->get()->map->toArray()->all();
}
```

O carregamento de 'questions' inclui o campo correctOptionIndex. Qualquer aluno autenticado obtém a resposta correta de todas as avaliações de todos os cursos com um GET em /api/quizzes. A nota em si não pode ser forjada, porque submitQuiz recalcula no servidor — esse é um ponto forte do projeto —, portanto o impacto recai sobre a integridade acadêmica, não sobre elevação de privilégio.

Arquivo: `backend-laravel/app/Services/LearningService.php:231-234`

```
public function listExercises(): array
{
    return PracticalExercise::query()->get()->map->toArray()->all();
}
```

Não há filtro por curso nem por acesso. É a mesma classe de problema do ISO-02, com conteúdo menos sensível: o enunciado, não a resposta. As submissões, por sua vez, são filtradas corretamente em listExerciseSubmissions.

Impacto

Fraude em avaliações: o gabarito fica disponível antes da tentativa.

Correcao sugerida

Omitir correctOptionIndex na listagem e devolvê-lo apenas na correção, dentro da resposta de submitQuiz, ou expor o gabarito somente aos perfis instrutor e admin.

Criterios de aceite

- [ ] `GET /api/quizzes` nao devolve `correctOptionIndex` para o papel student
- [ ] A correcao continua chegando ao aluno na resposta de `POST /api/quiz-submissions`
- [ ] Exercicios filtrados por curso acessivel
- [ ] Teste de feature cobrindo os dois pontos

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

[Seguranca] Emissao de certificado e configuracoes do sistema sem checagem adequada

Labels: security, media

Problema

PRIV-01 — A rota aceita qualquer usuário autenticado. Para o aluno o risco é contido: resolveActorUserId força o próprio identificador e a frequência mínima é recalculada no servidor (linhas 106 a 113), e ambas são proteções reais. O problema está no instrutor: ele informa qualquer userId e não existe verificação de InstructorScope sobre o curso, ao contrário do que o próprio projeto faz no download do PDF (CertificatePdfService.php:41-44). Um instrutor emite, portanto, certificado para aluno de curso que não leciona.

PRIV-02 — A leitura é pública — confirmado por curl, que devolve autoCertify, openEnrollment, allowGlobalChat, liveClassRecording, allowDirectMessages e autoArchiveDuration — e a escrita, restrita ao admin, faz merge de qualquer chave enviada, sem esquema algum. Hoje o conteúdo é apenas operacional; o risco é futuro e silencioso, porque qualquer campo sensível que um administrador passe a guardar ali nasce público.

Evidencia

Arquivo: `backend-laravel/routes/api.php:118-119 (POST /api/certificates) e app/Services/CertificateService.php:84-130`

```

```

```
Route::post('/certificates', [CertificateController::class, 'store'])
    ->middleware(['jwt', 'active']); // sem role:
// no service:
$userId = Identity::resolveActorUserId($requester, $input['userId'] ?? null);
```

A rota aceita qualquer usuário autenticado. Para o aluno o risco é contido: resolveActorUserId força o próprio identificador e a frequência mínima é recalculada no servidor (linhas 106 a 113), e ambas são proteções reais. O problema está no instrutor: ele informa qualquer userId e não existe verificação de InstructorScope sobre o curso, ao contrário do que o próprio projeto faz no download do PDF (CertificatePdfService.php:41-44). Um instrutor emite, portanto, certificado para aluno de curso que não leciona.

Arquivo: `backend-laravel/routes/api.php (GET /api/system-settings, sem middleware) e app/Services/SettingsService.php:16-40`

```
public function update(array $updates): array
{
    $merged = array_merge($row->data ?? [], $updates); // aceita chaves arbitrarias
```

A leitura é pública — confirmado por curl, que devolve autoCertify, openEnrollment, allowGlobalChat, liveClassRecording, allowDirectMessages e autoArchiveDuration — e a escrita, restrita ao admin, faz merge de qualquer chave enviada, sem esquema algum. Hoje o conteúdo é apenas operacional; o risco é futuro e silencioso, porque qualquer campo sensível que um administrador passe a guardar ali nasce público.

Impacto

Emissão de documento acadêmico por quem não tem relação com o curso, com registro de autoria indevida. Divulgação da configuração operacional e risco de exposição de dado sensível em alterações futuras.

Correcao sugerida

- **PRIV-01**: Acrescentar role:instructor,admin na rota e validar InstructorScope::courseIds no service quando o solicitante for instrutor e o userId pertencer a outra pessoa.
- **PRIV-02**: Exigir autenticação na leitura, ou devolver uma lista branca de chaves públicas, e validar quais chaves são aceitas na escrita.

Criterios de aceite

- [ ] `POST /api/certificates` exige papel e valida escopo do instrutor
- [ ] `GET /api/system-settings` exige autenticacao ou devolve apenas chaves publicas
- [ ] `PUT /api/system-settings` valida as chaves aceitas
- [ ] Teste de feature para cada regra

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

[Seguranca] Anotacoes do aluno manipulam HTML cru sem sanitizacao

Labels: security, baixa

Problema

O editor de anotações é contentEditable e o conteúdo trafega como HTML cru. A origem do dado é o localStorage do próprio usuário, na chave ava_student_lesson_notes, e nunca outro usuário ou o servidor — trata-se, portanto, de self-XSS: para explorar, a vítima precisa injetar em si mesma. O ponto que merece atenção é a exportação: o HTML vai para um arquivo .html baixado, que abre como página local com script ativo caso o conteúdo tenha sido colado de uma fonte hostil.

Evidencia

Arquivo: `src/components/StudentDashboard.tsx:316, :340 e :351`

```
noteEditorRef.current.innerHTML = savedNotes[activeLesson.id] || ''; // :316
const html = noteEditorRef.current.innerHTML; // :340 e :351
const htmlDoc = `<!DOCTYPE html>... ${html}</body></html>`; // exportacao
```

O editor de anotações é contentEditable e o conteúdo trafega como HTML cru. A origem do dado é o localStorage do próprio usuário, na chave ava_student_lesson_notes, e nunca outro usuário ou o servidor — trata-se, portanto, de self-XSS: para explorar, a vítima precisa injetar em si mesma. O ponto que merece atenção é a exportação: o HTML vai para um arquivo .html baixado, que abre como página local com script ativo caso o conteúdo tenha sido colado de uma fonte hostil.

Impacto

Baixo: a execução fica restrita à própria sessão. O arquivo exportado pode carregar script colado de terceiros.

Correcao sugerida

Sanitizar na leitura e na exportação, com lista branca de tags de formatação, ou substituir o contentEditable por marcação simples, como já é feito no conteúdo da aula (src/utils/lessonContent.ts).

Criterios de aceite

- [ ] Conteudo lido do localStorage passa por whitelist de tags antes do innerHTML
- [ ] Arquivo exportado nao contem `<script>` nem manipuladores inline
- [ ] Teste cobrindo payload de script salvo nas anotacoes

--- FIM ISSUE 8 ---